Министерство образования Республики Беларусь
Учреждение образования
Белорусский Государственный университет информатики и радиоэлектроники

Инженерно-экономический факультет
Кафедра экономической информатики

Отчет

по лабораторной работе № 3

на тему «Реализация базовой структуры системы и GraphQL API»

Выполнил: Летко А.С.
студент группы 377901

Проверила: Сильванович Ю.В.

Минск 2026

ЦЕЛЬ РАБОТЫ

Реализовать базовую структуру распределённой платёжной системы и внешний GraphQL API для получения данных, подготовить внутренние сервисы и их хранилища, а также организовать взаимодействие компонентов с помощью mock-клиентов.

1 СТРУКТУРА ПРОЕКТА

В качестве предметной области использована платёжная система, спроектированная в предыдущей лабораторной работе. Внешний интерфейс реализован на Java 17 с применением Spring Boot и Spring GraphQL. Для хранения данных используется PostgreSQL, для сборки – Maven, для запуска компонентов – Docker Compose.

Проект разделён на модуль gateway и девять модулей внутренних сервисов: identity-service, merchant-service, payment-service, paymentmethod-service, account-service, refund-service, transaction-service, notification-service и analytics-service. В каждом сервисном модуле выделены классы репозитория и сервисного слоя. Общую структуру проекта следует представить на рисунке 1.1.

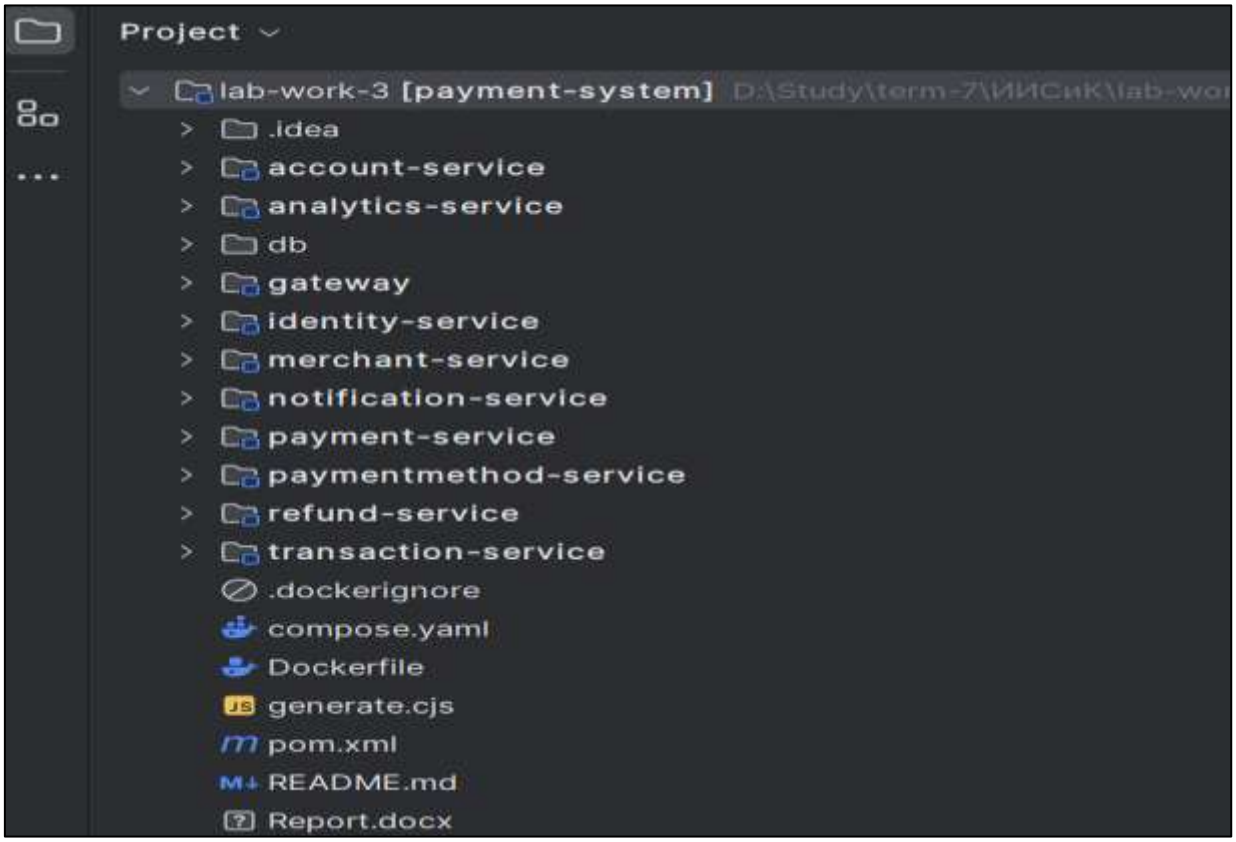


Рисунок 1.1 – Структура модулей проекта

Gateway предоставляет GraphQL API и подключает сервисные модули как зависимости Maven. Репозиторий конкретного модуля выполняет SQL-запросы только к таблице своего сервиса, а сервисный слой предоставляет операции получения всех записей и поиска по идентификатору. Для передачи записей в учебной реализации применяются отображения Map<String, Object>.

На данном этапе сервисы являются отдельными Maven-модулями и работают в одном процессе вместе с gateway. Они не являются отдельными запущенными микросервисами. Такое построение позволяет подготовить границы компонентов до внедрения реальных сетевых взаимодействий.

1.1 Назначение компонентов системы

Identity Service отвечает за сведения о пользователях. В учебной реализации в его хранилище представлены идентификатор, имя, адрес электронной почты и роль пользователя. Операция user позволяет получить эти данные по идентификатору. Регистрация и аутентификация в рамках рассматриваемой версии не выполняются.

Merchant Service содержит сведения об организациях, принимающих платежи. В записи мерчанта указываются его идентификатор, идентификатор связанного пользователя, наименование и статус. Идентификатор пользователя служит логической связью с данными Identity Service.

Payment Service хранит сведения о платежах и является центральным компонентом рассматриваемого сценария чтения. Запись платежа содержит идентификаторы клиента, мерчанта и платёжного метода, сумму, валюту, статус, описание и дату создания. Доступны получение конкретного платежа и просмотр списка платежей.

Payment Method Service содержит данные платёжных методов: тип, последние четыре цифры, статус и идентификатор клиента. Account Service хранит сведения о владельце счёта, его балансе, валюте и статусе. Данные счетов подготовлены для дальнейшего развития системы; отдельной операции получения счёта в текущей схеме GraphQL нет.

Refund Service хранит данные о возвратах по платежам. Transaction Service содержит записи финансовых операций, связанных с клиентом и платежом. Через GraphQL можно получить возвраты по идентификатору платежа и транзакции с отбором по клиенту или платежу.

Notification Service содержит подготовленные сведения об уведомлениях. Analytics Service хранит демонстрационные агрегированные показатели мерчанта за определённые периоды. Уведомления и статистика не формируются автоматически из событий: необходимые записи добавлены скриптом инициализации.

1.2 Разделение проекта на слои

Слой доступа к данным представлен классами Repository. Каждый репозиторий получает JdbcTemplate и использует параметризованный запрос при поиске записи по идентификатору. Применение параметра позволяет передавать значение отдельно от текста SQL-запроса.

Сервисный слой обращается к своему репозиторию и предоставляет операции list и find. Результаты чтения представлены коллекциями Map: ключам соответствуют имена столбцов, а значения содержат прочитанные из PostgreSQL данные. Отдельные классы доменных моделей и DTO в данной версии не выделены.

Внешний слой отвечает за приём GraphQL-запросов и формирование ответа. Он обращается к сервисам через абстракцию клиента. При этом логика отбора, сортировки и преобразования имён полей в текущем прототипе находится в QueryController; перенос этой логики в соответствующие сервисы возможен при дальнейшем развитии проекта.

2 ОРГАНИЗАЦИЯ ХРАНЕНИЯ ДАННЫХ

В Docker Compose запускаются контейнер PostgreSQL и контейнер приложения. В одной базе данных payments созданы отдельные схемы identity, merchant, payment, paymentmethod, account, refund, transaction, notification и analytics. Каждая схема относится к соответствующему сервису. В рамках текущей реализации используется общий экземпляр базы и общая учётная запись, а не отдельные серверы баз данных.

При первом создании тома PostgreSQL автоматически выполняется скрипт db/init.sql. Он создаёт схемы и таблицы, а затем добавляет демонстрационные записи. В частности, платёж p1 связан с клиентом u1, мерчантом m1 и платёжным методом pm1; для него подготовлены транзакции и возврат. Это позволяет проверить получение связанных данных через GraphQL.

Пример обращения PaymentRepository к собственным данным приведён в листинге 2.1. Запрос получает платёж по идентификатору из таблицы payment.payments; если запись отсутствует, сервис возвращает null.

Листинг 2.1 – Запрос данных платежа по идентификатору:

```
select id, client_id, merchant_id, payment_method_id,
       amount, currency, status, description, created_at
from payment.payments where id = ?
```

Связи между записями разных сервисов представлены идентификаторами. Межсервисные внешние ключи в базе не создаются: например, client_id в платеже указывает на пользователя логически. Проверку существующих схем и таблиц можно представить на рисунке 2.1.

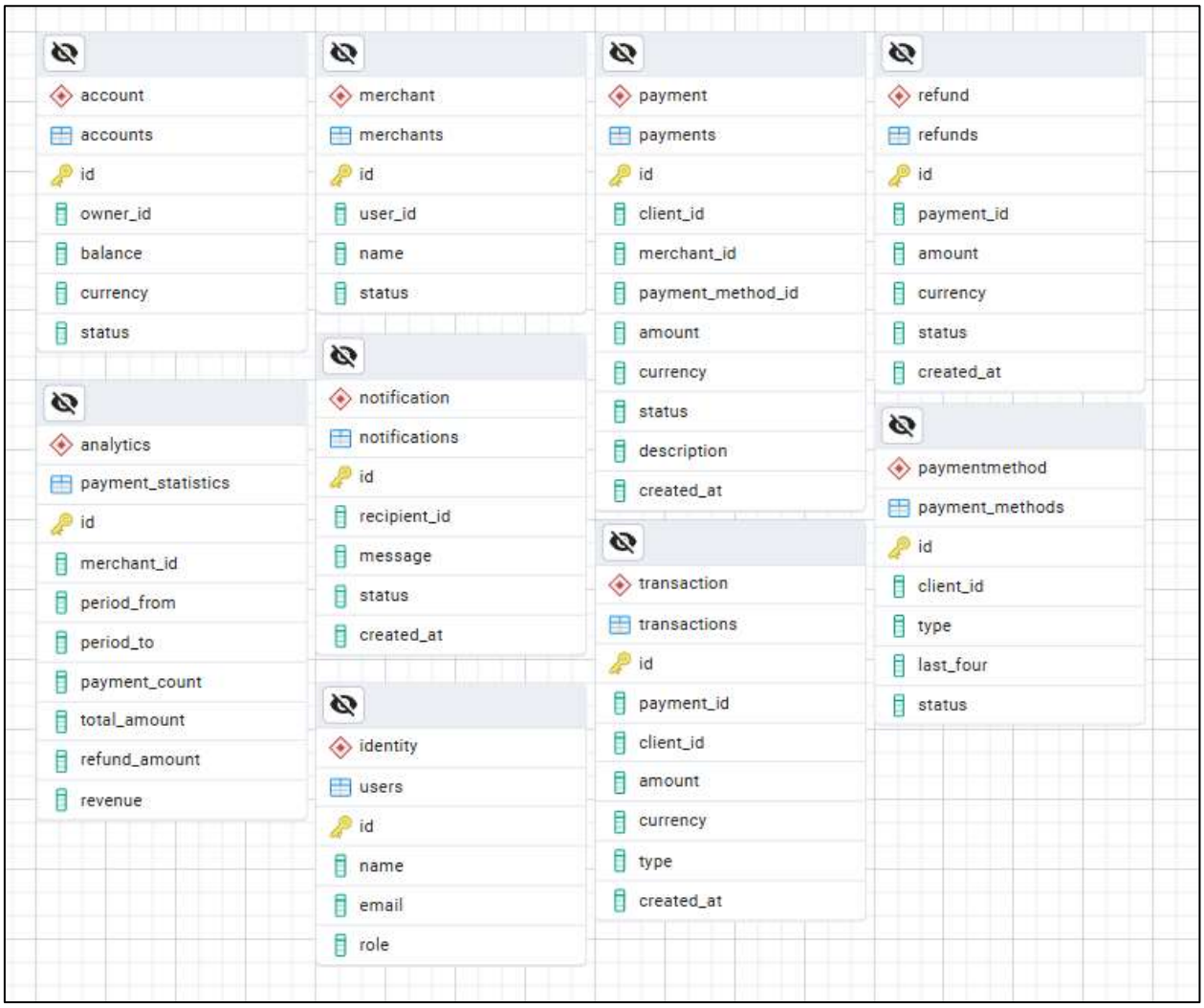


Рисунок 2.1 – Схемы базы данных платёжной системы

2.1 Состав таблиц и демонстрационные данные

Для каждого сервисного модуля подготовлена собственная таблица: users, merchants, payments, payment_methods, accounts, refunds, transactions, notifications или payment_statistics. Таблицы расположены в одноимённых схемах своих сервисов. Первичные ключи в учебном наборе представлены строковыми идентификаторами, например u1 и p1.

В начальном наборе данных содержатся два пользователя, один мерчант и один платёжный метод. Добавлены два платежа: p1 со статусом SUCCESS на сумму 125,50 BYN и p2 со статусом CREATED на сумму 45,00 BYN. Для платежа p1 созданы возврат на сумму 25,00 BYN и две транзакции: PAYMENT и REFUND.

Для демонстрации аналитики добавлены две записи статистики по мерчанту m1 – за январь и февраль 2026 года. Январская запись содержит количество платежей 1, общий объём 125,50, объём возвратов 25,00 и

показатель revenue 100,50. Это подготовленные значения: приложение не пересчитывает их по таблицам платежей и возвратов.

2.2 Работа с хранилищем

PostgreSQL сохраняет данные в именованном томе Docker. Поэтому обычная остановка контейнеров не удаляет созданные записи. Скрипт init.sql выполняется контейнером при инициализации нового каталога данных; последующие запуски с существующим томом используют уже созданное содержимое базы.

Все модули подключаются к одной базе и используют одну техническую учётную запись. Разделение обеспечивается пространствами имён и тем, к каким таблицам обращается код репозиториев; самостоятельной настройки отдельных прав доступа и отдельных баз для каждого сервиса в этой реализации нет.

В таблицах суммы хранятся в PostgreSQL как numeric(15,2), а даты в учебной модели – как строки. Для корректного строкового сравнения демонстрационные значения дат записаны в согласованном формате. В дальнейшем для надёжной обработки дат целесообразно использовать специализированные типы базы данных.

3 GRAPHQL API И MOCK-ВЗАИМОДЕЙСТВИЕ

GraphQL API размещён в модуле gateway. Схема находится в файле gateway/src/main/resources/graphql/schema.graphqls. В ней определены типы User, Payment, PaymentMethod, Transaction, Refund, Analytics и другие, а также входные типы фильтрации, сортировки и диапазона дат.

В соответствии с предыдущей работой реализованы операции чтения user, payment, payments, transactions, paymentMethods, refunds, analytics и reports. Запрос payment возвращает объект по идентификатору, payments – список с возможностью фильтрации и сортировки, остальные операции предоставляют связанные и аналитические данные. Пример запроса приведён в листинге 3.1.

Листинг 3.1 – Получение платежа и связанных данных:

```
query {
  payment(id: "p1") {
    id amount currency status
    client { id name }
    merchant { id name }
    refunds { id amount status }
  }
}
```

Контроллер QueryController принимает запрос и обращается к интерфейсу ServiceClient. Реализация MockServiceClient перенаправляет вызовы соответствующим сервисным классам в пределах одного процесса.

Таким образом, межсервисное сетевое взаимодействие на текущем этапе имитируется, реальные gRPC-вызовы и Kafka не используются. При дальнейшем развитии интерфейс клиента можно реализовать с использованием сетевого транспорта.

Получение связанных данных организовано отдельными обработчиками полей Payment: клиент, мерчант, платёжный метод, возвраты и транзакции запрашиваются через интерфейс клиента. При этом в текущем QueryController также выполняются фильтрация, сортировка и отбор аналитических записей; это упрощение учебной реализации, которое в дальнейшем можно перенести в сервисные слои.

Для операций чтения отсутствующий объект возвращается как null, а список без совпадений – как пустой массив. В данной версии суммы представлены типом GraphQL Float, а даты – строками; для промышленной платёжной системы необходимы точный десятичный тип и строгая обработка дат. Mutation, авторизация и обработка событий в объём этой работы не включены.

3.1 Состав запросов GraphQL

Операция user(id) возвращает пользователя, а payment(id) – платёж. Обязательность аргумента идентификатора обозначается в схеме символом «!». Если объект не найден, результат соответствующего поля равен null, поскольку возвращаемый тип допускает отсутствие значения.

Операция payments принимает необязательные объекты filter и sort. В PaymentFilterInput предусмотрены статус, границы периода и суммы. PaymentSortInput задаёт поле CREATED_AT или AMOUNT и направление ASC либо DESC. При отсутствии фильтра возвращаются все подготовленные платежи.

Операция transactions допускает отбор по paymentId и clientId, paymentMethods выбирает методы обязательного клиента clientId, а refunds – возвраты с необязательным ограничением по paymentId. Для аналитических операций analytics и reports требуются merchantId и диапазон дат period с полями from и to.

Связанные поля позволяют в рамках одного запроса получить платёж, сведения о клиенте и мерчанте, платёжный метод, возвраты и транзакции. Клиент сам указывает необходимые поля, поэтому ответ содержит только запрошенные данные. Например, при запросе только id и status поля amount и description в ответ не включаются.

3.2 Последовательность обработки запроса

Сначала пользователь вводит запрос на локальной странице приложения и отправляет его на адрес /graphql. GraphQL-подсистема проверяет запрос по

схеме: существование полей и соответствие аргументов объявленным типам. Затем вызывается соответствующий метод контроллера.

Контроллер обращается к ServiceClient. MockServiceClient определяет нужный сервис и вызывает его метод. Сервис передаёт запрос репозиторию, а репозиторий читает данные из своей схемы PostgreSQL. Полученные записи возвращаются обратно через сервис и клиент к GraphQL-контроллеру.

Имена столбцов SQL в формате snake_case преобразуются контроллером в имена полей camelCase, используемые схемой GraphQL. Если клиент запросил связанные поля платежа, для них выполняются дополнительные обращения через тот же интерфейс клиента. После этого GraphQL формирует ответ в формате JSON.

3.3 Границы текущей реализации

Mock-клиент в данном проекте не возвращает отдельный набор заранее зашитых в клиент ответов: он локально делегирует вызовы реальным сервисным классам, которые читают PostgreSQL. Следовательно, данные ответа берутся из базы, тогда как транспорт между gateway и сервисами заменён локальным вызовом.

Для учебного прототипа фильтрация выполняется после чтения списка записей в память. Это удобно на небольшом демонстрационном наборе, но при увеличении объёма данных потребует переноса условий в репозиторий, добавления постраничного чтения и оптимизации количества запросов к связанным данным.

Планируемый в архитектуре переход к отдельным процессам потребует определить сетевые контракты и заменить реализацию клиента, а также отдельно решить вопросы развёртывания, доступа к данным и обработки ошибок сети. Эти механизмы не следует считать уже реализованными в лабораторной работе.

4 ЗАПУСК И ПРОВЕРКА РАБОТЫ

4.1 Запуск компонентов

Для запуска необходимо открыть терминал в корне проекта и выполнить команду docker compose up --build. Docker Compose собирает Java-приложение, создаёт контейнер PostgreSQL и ожидает его готовности перед запуском приложения. При первом запуске загружаются необходимые образы и зависимости Maven.

```
docker compose up --build
```

После запуска приложение доступно по адресу http://localhost:18080/. На странице расположен редактор GraphQL-запросов и область ответа. Результат запуска контейнеров следует представить на рисунке 4.1.



Рисунок 4.1 – Запуск приложения и базы данных в Docker

4.2 Получение объекта и связанных данных

Для проверки получения данных по идентификатору в редакторе выполняется запрос из листинга 3.1. Для платежа p1 ожидаются статус SUCCESS, сумма 125.50 BYN, данные клиента и мерчанта, а также возврат. Запрос и ответ следует показать на рисунке 4.2.

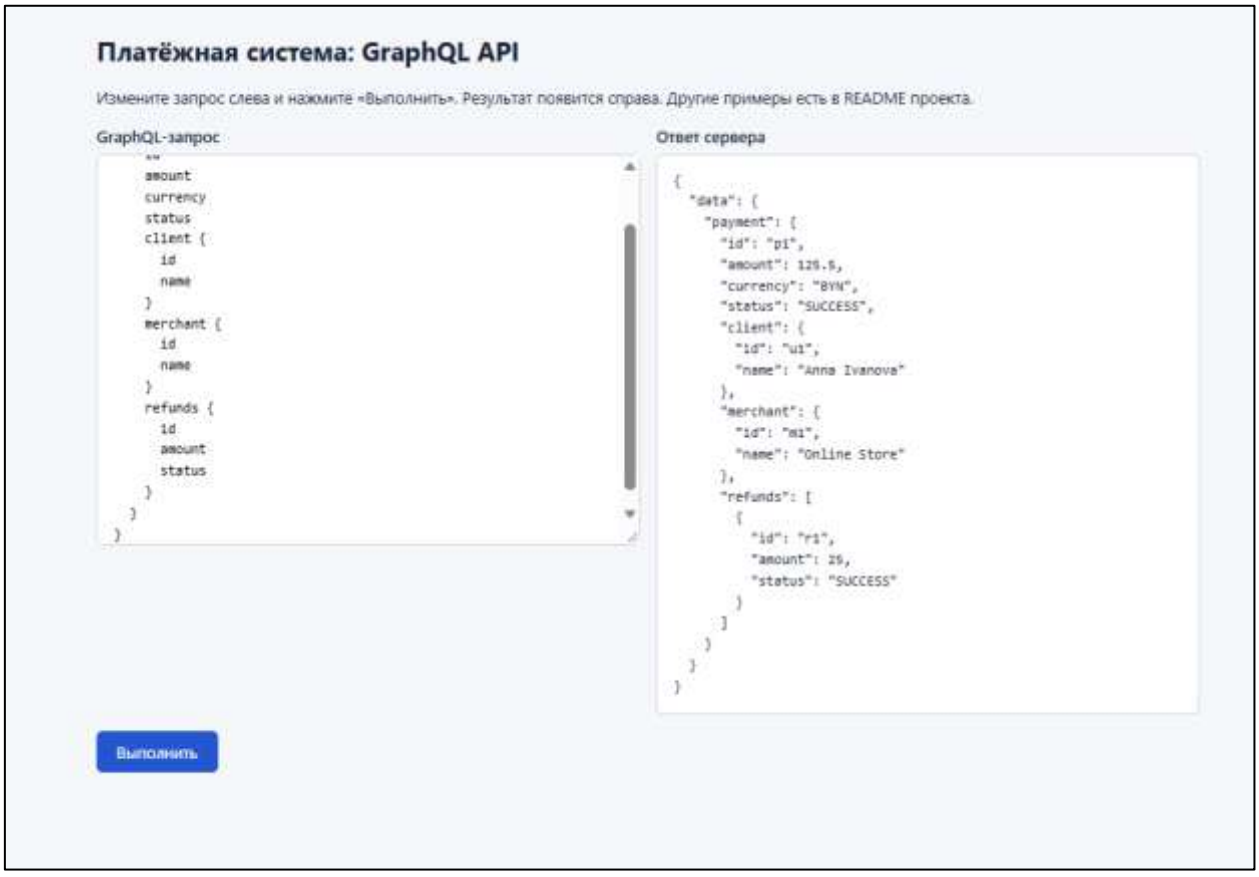


Рисунок 4.2 – Получение платежа и связанных данных через GraphQL

4.3 Проверка списка и фильтрации

Операция payments поддерживает фильтрацию по статусу, датам и сумме, а также сортировку. Запрос из листинга 4.1 выбирает успешные платежи и сортирует их по сумме по убыванию. Для подготовленных данных в ответе должен присутствовать платёж p1. Пример результата следует показать на рисунке 4.3.

Листинг 4.1 – Получение списка платежей с фильтрацией и сортировкой:

```
query {
  payments(filter: {status: SUCCESS},
           sort: {field: AMOUNT, direction: DESC}) {
    id amount currency status
  }
}
```

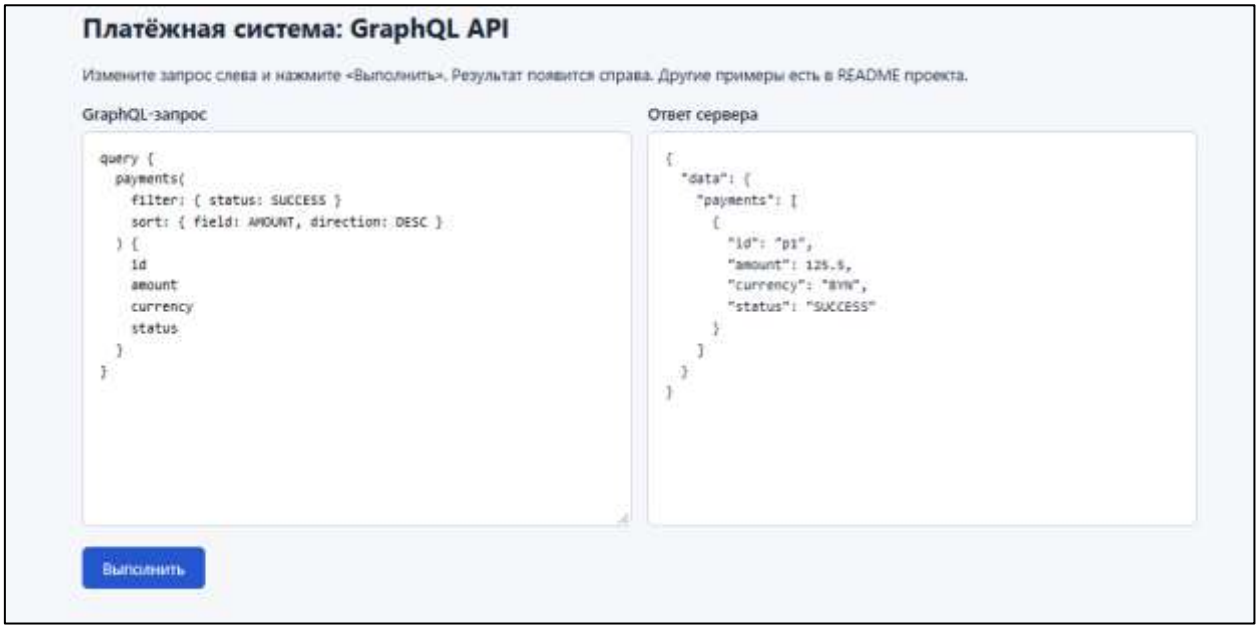


Рисунок 4.3 – Результат фильтрации списка платежей

4.4 Отсутствующие данные и аналитика

При обращении payment(id: "missing") отсутствующий платёж возвращается как null. При запросе возвратов с неизвестным paymentId возвращается пустой список. Такая проверка показывает, что отсутствие данных не приводит к аварийному завершению запроса.

Листинг 4.2 – Проверка отсутствующих данных:

```
query {
  payment(id: "missing") { id status }
  refunds(paymentId: "missing") { id }
}
```

Для получения аналитики используется операция analytics с идентификатором мерчанта и периодом. Отчётные записи подготавливаются заранее и выбираются по пересечению периодов. Analytics возвращает первую подходящую запись, а reports – список подходящих записей; автоматический пересчёт показателей в текущей версии не реализован.

Листинг 4.3 – Получение демонстрационных аналитических показателей:

```
query {
  analytics(merchantId: "m1",
    period: {from: "2026-01-01", to: "2026-01-31"}) {
    paymentCount totalAmount refundAmount revenue
  }
}
```

Результаты запроса отсутствующих данных и аналитики следует представить на рисунке 4.4.

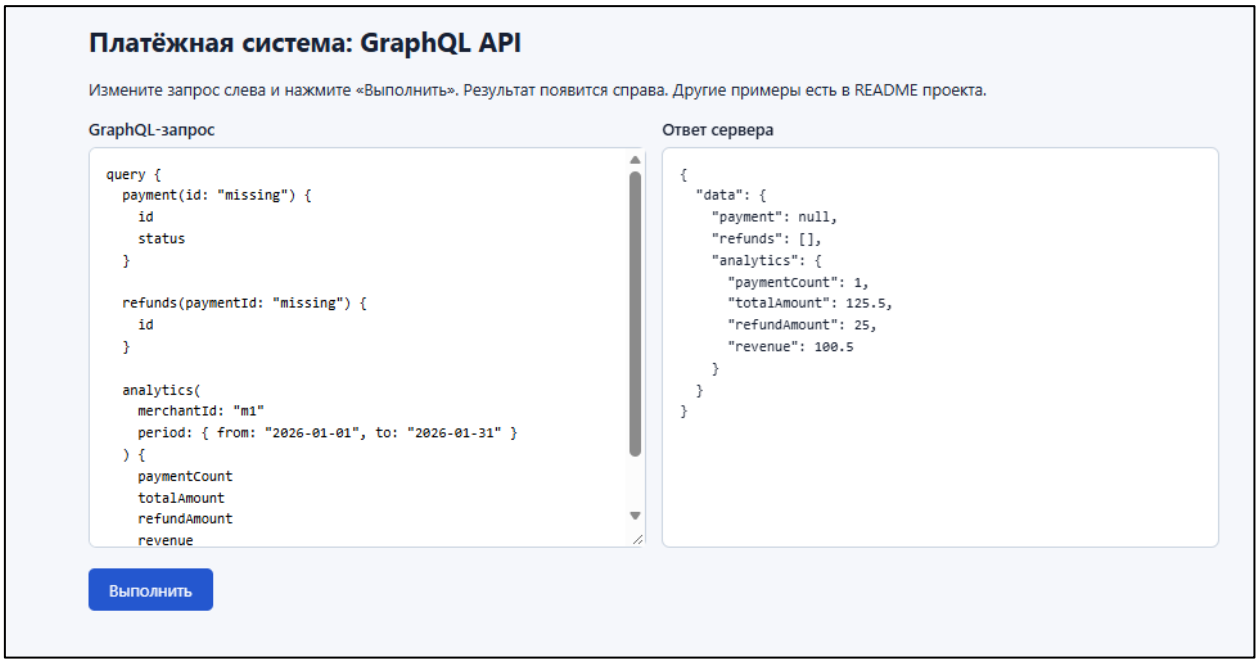


Рисунок 4.4 – Проверка отсутствующих данных и аналитических показателей

4.5 Дополнительные сценарии проверки

Помимо поиска отдельного платежа, можно проверить получение всех транзакций клиента u1 с помощью запроса transactions(clientId: "u1"). Для подготовленных данных возвращаются записи t1 и t2, относящиеся к платежу p1. При передаче одновременно paymentId и clientId запись должна соответствовать обоим заданным условиям.

Для проверки списка платёжных методов используется запрос paymentMethods(clientId: "u1"). В ответе ожидается метод pm1 типа CARD с последними четырьмя цифрами 4242. Полный номер карты в демонстрационной записи отсутствует.

При запросе reports для мерчанта m1 и диапазона с 1 января по 31 декабря 2026 года выбираются две подготовленные записи – за январь и февраль. Условие отбора учитывает пересечение периода записи и

запрошенного диапазона. При отсутствии подходящих отчётных записей возвращается пустой список.

Проверка ошибок входного запроса выполняется средствами GraphQL. Например, если запросить поле, не объявленное в типе Payment, сервер вернёт сообщение об ошибке проверки схемы. Если же запрос корректен, но идентификатор не найден, поле payment вернёт null. Эти ситуации имеют разный смысл: первая связана с формой запроса, вторая – с отсутствием записи.

4.6 Остановка и повторный запуск

Если приложение запущено в текущем окне терминала, его можно остановить сочетанием Ctrl+C. Команда docker compose down удаляет контейнеры и сеть проекта, но сохраняет именованный том базы. Для повторного запуска используется та же команда docker compose up --build.

Команда docker compose down -v удаляет также том с данными. После следующего запуска база будет создана заново и заполнена исходными демонстрационными записями. Эту команду следует применять, когда необходимо именно восстановить начальное состояние, поскольку изменения данных в томе будут потеряны.

5 ВЫПОЛНЕНИЕ ПУНКТОВ ЗАДАНИЯ

В данном разделе результаты работы сопоставлены с пятью пунктами задания. Снимки экрана позволяют показать расположение компонентов и подтвердить выполнение запросов на подготовленном наборе данных.

5.1 Базовая структура проекта

В корневом Maven-проекте определены модуль gateway и девять сервисных модулей. Gateway содержит конфигурацию приложения, схему GraphQL, контроллер запросов и абстракцию клиента. Модули внутренних сервисов включают классы сервисного слоя и репозиториев. Корневой pom.xml объединяет модули при сборке. На рисунке 5.1 следует показать состав модулей в этом файле.



Рисунок 5.1 – Модули в конфигурации Maven

Базовая конфигурация приложения задаёт подключение к PostgreSQL и порт внутри контейнера, а compose.yaml описывает контейнеры и переменные окружения. Проверка готовности PostgreSQL используется как условие запуска gateway. Конфигурацию контейнеров следует показать на рисунке 5.2.



Рисунок 5.2 – Конфигурация приложения и PostgreSQL в Docker Compose

5.2 Внутренние сервисы и данные

В качестве примера внутреннего сервиса рассмотрен Payment Service. Класс PaymentRepository использует JdbcTemplate для чтения списка записей и поиска платежа по идентификатору. Класс PaymentService предоставляет

методы list и find. Эти классы и их расположение в пакете следует показать на рисунке 5.3.



Рисунок 5.3 – Сервисный слой и репозиторий платежей

Модель хранимого платежа выражена структурой таблицы payment.payments, а модель внешнего ответа – типом Payment в GraphQL. Записи в Java передаются как Map<String, Object>. Отдельные типизированные Java-классы моделей данных в текущем проекте не реализованы; этот пункт задания выполнен в упрощённом виде. Структуру таблицы и подготовленные записи следует показать на рисунке 5.4.

```
CREATE SCHEMA identity;
CREATE SCHEMA merchant;
CREATE SCHEMA payment;
CREATE SCHEMA paymentmethod;
CREATE SCHEMA account;
CREATE SCHEMA refund;
CREATE SCHEMA transaction;
CREATE SCHEMA notification;
CREATE SCHEMA analytics;
CREATE TABLE identity.users (id varchar PRIMARY KEY, name varchar NOT NULL, email varchar NOT NULL, role varchar NO
CREATE TABLE merchant.merchants (id varchar PRIMARY KEY, user_id varchar NOT NULL, name varchar NOT NULL, status va
CREATE TABLE payment.payments (id varchar PRIMARY KEY, client_id varchar NOT NULL, merchant_id varchar NOT NULL, pa
CREATE TABLE paymentmethod.payment_methods (id varchar PRIMARY KEY, client_id varchar NOT NULL, type varchar NOT NU
CREATE TABLE account.accounts (id varchar PRIMARY KEY, owner_id varchar NOT NULL, balance numeric(15,2) NOT NULL, c
CREATE TABLE refund.refunds (id varchar PRIMARY KEY, payment_id varchar NOT NULL, amount numeric(15,2) NOT NULL, cu
CREATE TABLE transaction.transactions (id varchar PRIMARY KEY, payment_id varchar NOT NULL, client_id varchar NOT N
CREATE TABLE notification.notifications (id varchar PRIMARY KEY, recipient_id varchar NOT NULL, message varchar NOT
CREATE TABLE analytics.payment_statistics (id varchar PRIMARY KEY, merchant_id varchar NOT NULL, period_from varcha
INSERT INTO identity.users VALUES ('u1','Anna Ivanova','anna@example.com','CLIENT'),('u2','Pavel Petrov','pavel@exa
INSERT INTO merchant.merchants VALUES ('m1','u2','Online Store','ACTIVE');
INSERT INTO paymentmethod.payment_methods VALUES ('pm1','u1','CARD','4242','ACTIVE');
INSERT INTO account.accounts VALUES ('a1','u1',500.00,'BYN','ACTIVE'),('a2','u2',125.50,'BYN','ACTIVE');
INSERT INTO payment.payments VALUES ('p1','u1','m1','pm1',125.50,'BYN','SUCCESS','Order 1001','2026-01-10T12:00:00Z
INSERT INTO refund.refunds VALUES ('r1','p1',25.00,'BYN','SUCCESS','2026-01-12T09:00:00Z');
INSERT INTO transaction.transactions VALUES ('t1','p1','u1',125.50,'BYN','PAYMENT','2026-01-10T12:01:00Z'),('t2','p
INSERT INTO notification.notifications VALUES ('n1','u1','Payment completed','SENT','2026-01-10T12:01:00Z');
INSERT INTO analytics.payment_statistics VALUES ('s1','m1','2026-01-01','2026-01-31',1,125.50,25.00,100.50),('s2','
```

Рисунок 5.4 – Структура таблицы платежей и начальные данные

Разделение данных реализовано схемами PostgreSQL в одной базе payments. Код PaymentRepository обращается к payment.payments; аналогично остальные репозитории работают со своими таблицами. Это логическое разделение в рамках общего экземпляра PostgreSQL и общей учётной записи.

5.3 GraphQL schema и Query

Схема gateway/src/main/resources/graphql/schema.graphqls определяет корневой тип Query, типы ответов, аргументы, перечисления и входные структуры. Для лабораторной работы реализованы операции чтения; операции изменения состояния в схему не включены. Состав Query и тип Payment следует показать на рисунке 5.5.

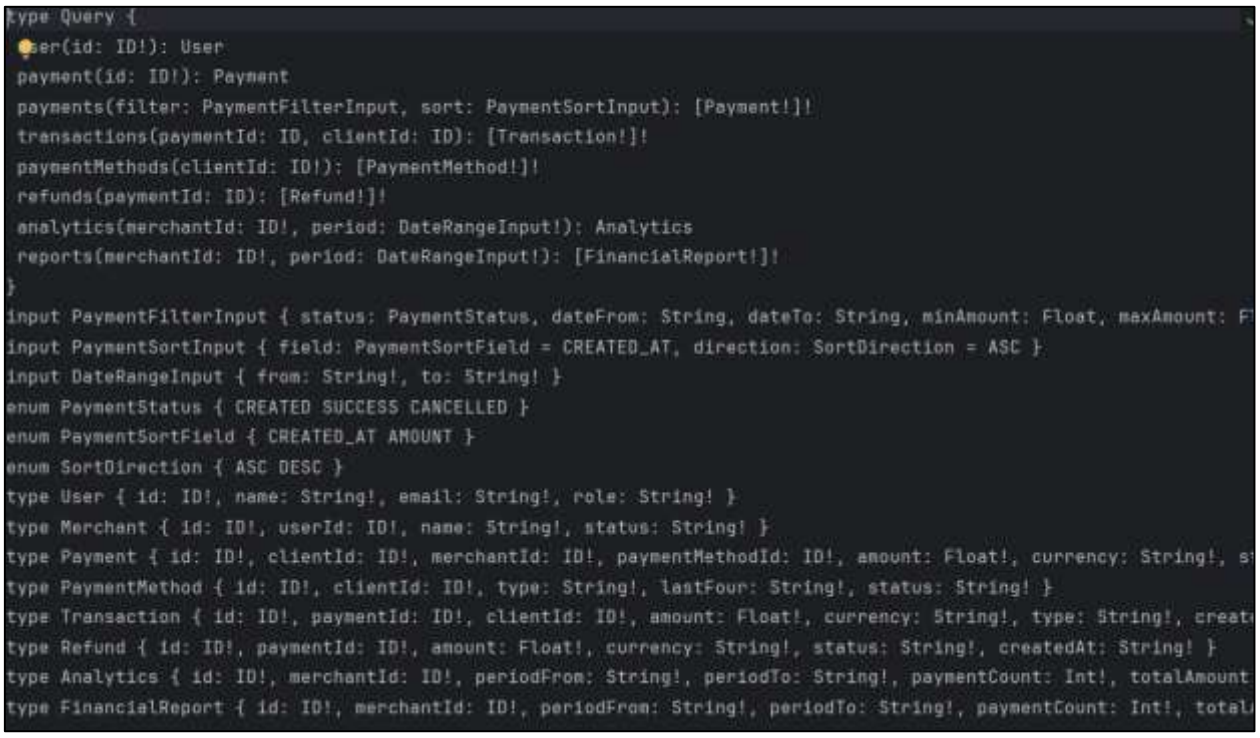


Рисунок 5.5 – Схема GraphQL и доступные запросы

Для проверки пользователя по идентификатору можно выполнить приведённый ниже запрос. Пользователь u1 должен иметь имя Anna Ivanova и роль CLIENT. На рисунке 5.6 следует показать одновременно запрос и полученный ответ.

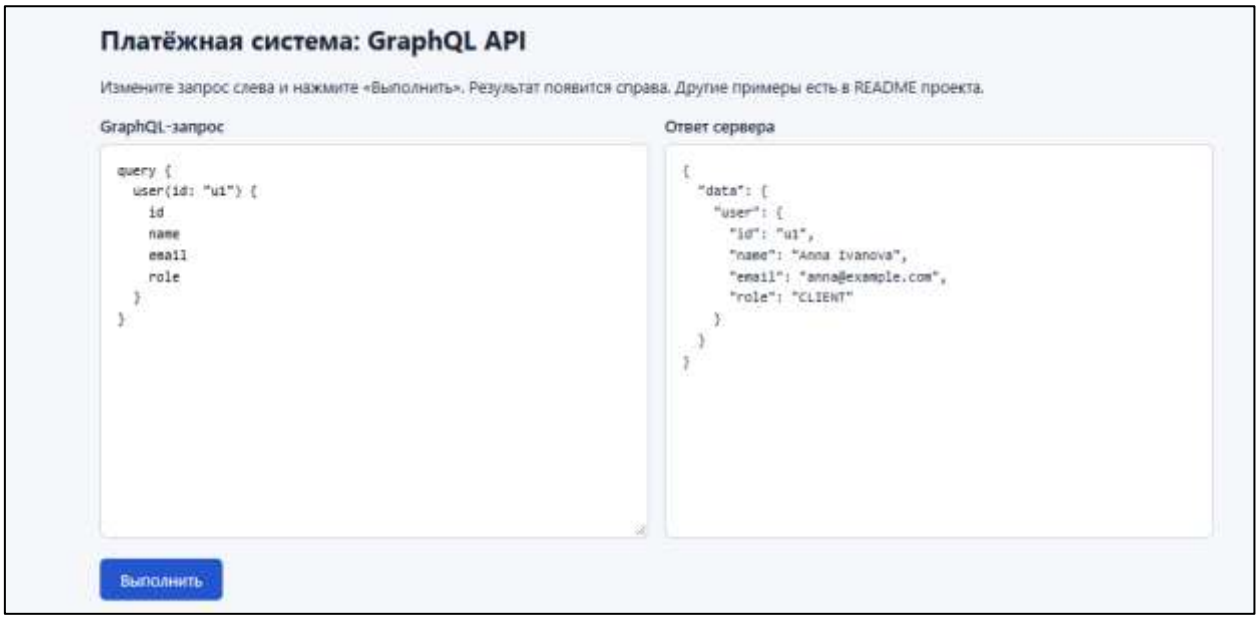


Рисунок 5.6 – Получение пользователя по идентификатору

5.4 Mock-взаимодействие компонентов

Gateway обращается к данным через интерфейс ServiceClient, в котором определены операции list и find. Его реализация MockServiceClient выбирает нужный сервис и вызывает его непосредственно в процессе приложения. На рисунке 5.7 следует показать интерфейс, а на рисунке 5.8 – реализацию.

```
public interface ServiceClient {  3 usages  1 implementation
 List<Map<String,Object>> list(String service);  1 implementation
 Map<String,Object> find(String service, String id);  1 usage  1 implementation
}
```

Рисунок 5.7 – Интерфейс клиента внутренних сервисов

```
@Component
public class MockServiceClient implements ServiceClient {
 private final IdentityService identity;  3 usages
 private final MerchantService merchant;  3 usages
 private final PaymentService payment;  3 usages
 private final PaymentmethodService paymentmethod;  3 usages
 private final AccountService account;  3 usages
 private final RefundService refund;  3 usages
 private final TransactionService transaction;  3 usages
 private final NotificationService notification;  3 usages
 private final AnalyticsService analytics;  3 usages
 public MockServiceClient(IdentityService identity, MerchantService merchant, PaymentService payment, Paymentmet
 public List<Map<String,Object>> list(String service) { return switch(service) { case "identity" -> identity.li
 public Map<String,Object> find(String service,String id) { return switch(service) { case "identity" -> identity
```

Рисунок 5.8 – Mock-реализация клиента внутренних сервисов

MockServiceClient заменяет транспортный уровень и обращается к реальным сервисным классам в том же процессе. Сервисы затем читают данные из PostgreSQL. Таким образом, ответ не является жёстко заданным объектом внутри mock-клиента; реальные gRPC-вызовы в проекте не выполняются.

5.5 Базовые операции чтения и обработка отсутствия данных

Получение списка платежей, отдельного объекта, связанных данных и пустого результата можно проверить отдельными запросами. Например, запрос ниже получает платёж p1 и сведения о платёжном методе, возврате и транзакциях. Ожидаемый результат следует показать на рисунке 5.9.

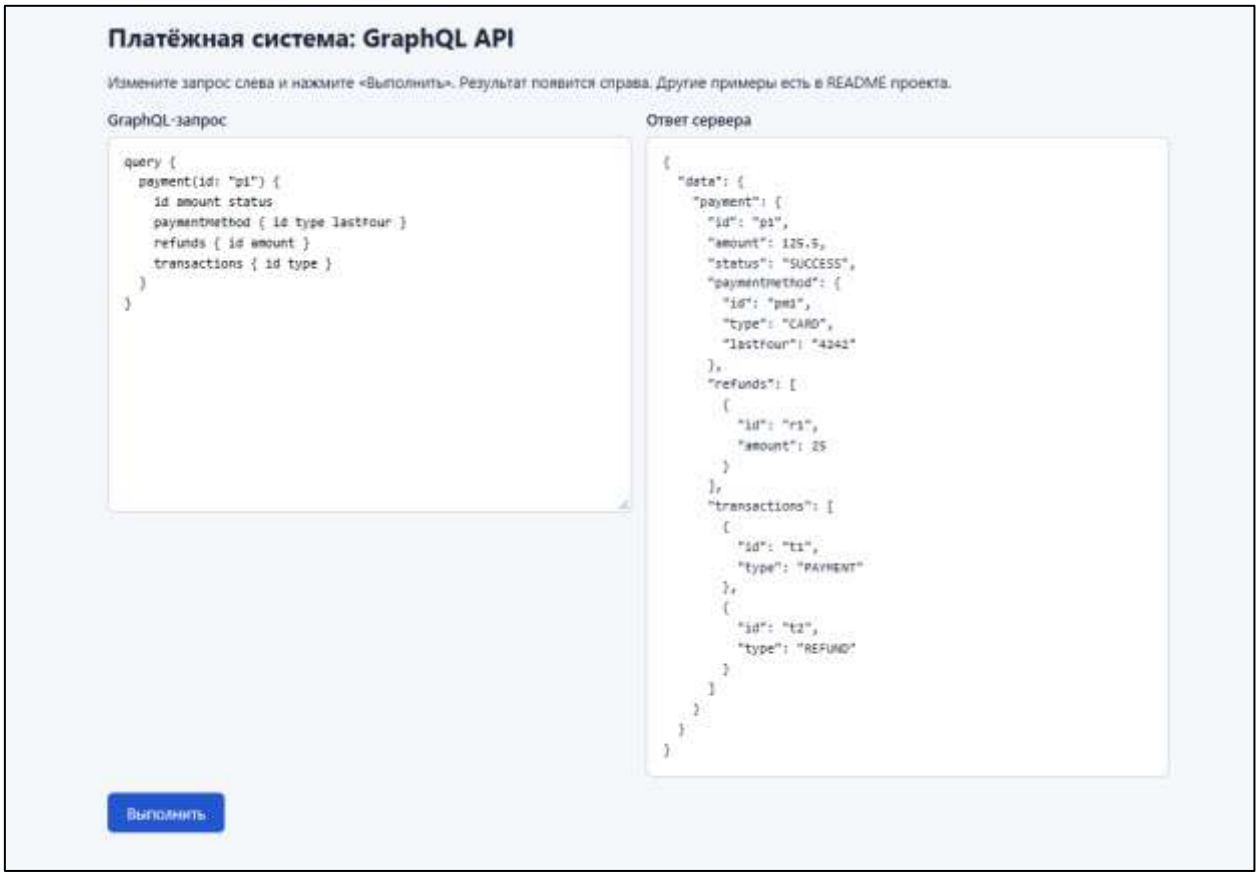


Рисунок 5.9 – Получение платежа и связанных сущностей

Запрос отсутствующего платежа возвращает null, а список возвратов для отсутствующего идентификатора – пустой массив. Это поведение определяется допускающим null типом поля payment и возвращением пустого списка для refunds. Результат проверки следует показать на рисунке 5.10.

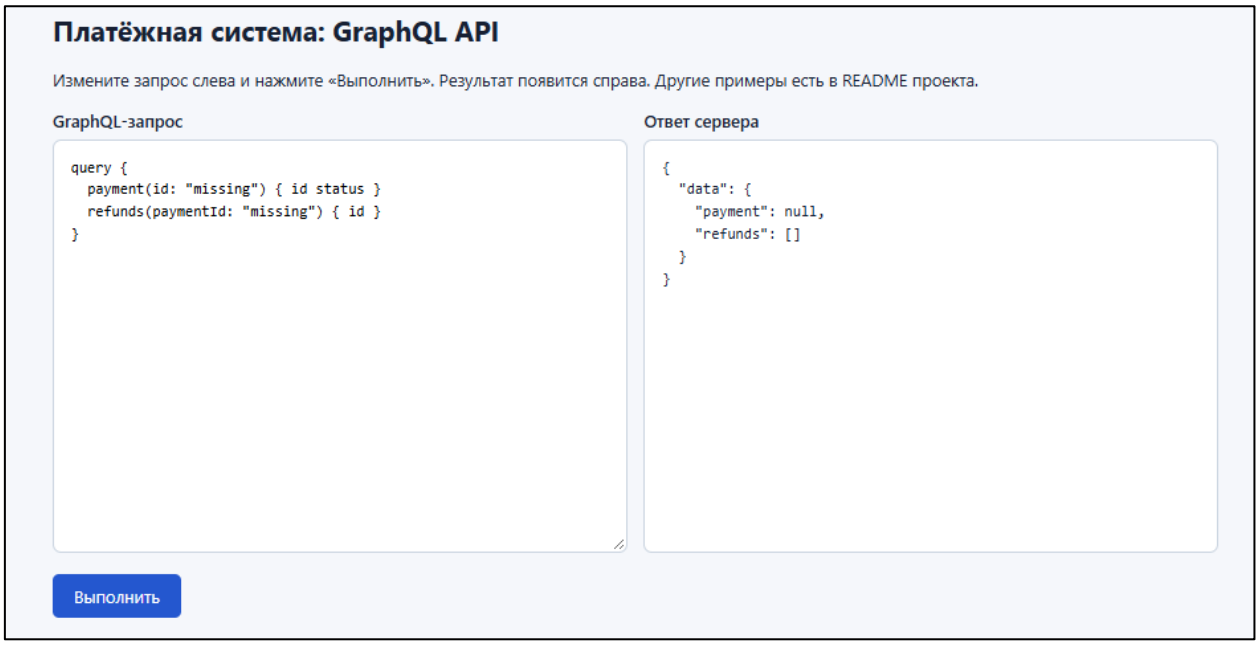


Рисунок 5.10 – Проверка обработки отсутствующих данных

Отдельно можно получить историю операций клиента и его платёжные методы. В подготовленном наборе для u1 имеются транзакции t1 и t2 и платёжный метод pm1. Запрос и ответ следует показать на рисунке 5.11.

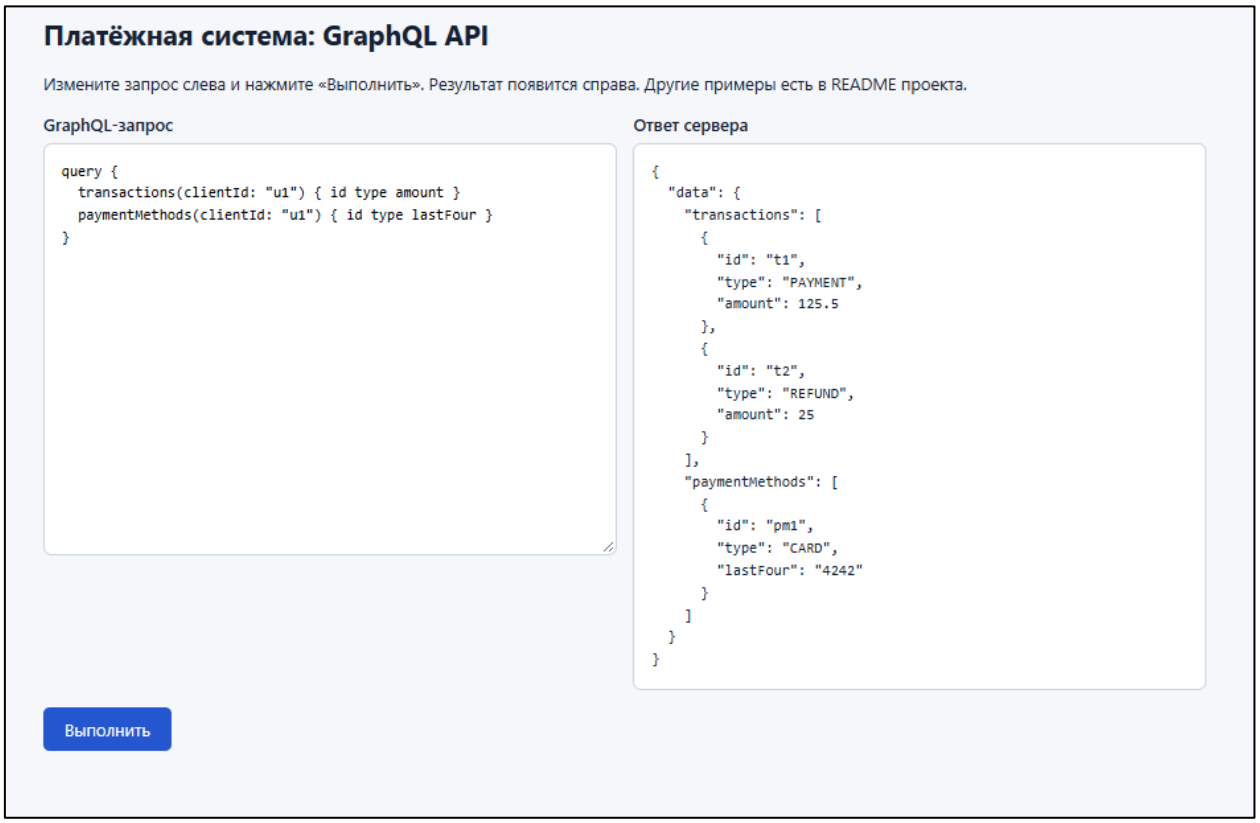


Рисунок 5.11 – Получение транзакций и платёжных методов клиента

Для проверки аналитических запросов можно получить январскую статистику мерчанта m1. В ответе ожидаются количество платежей 1, общий объём 125.5, сумма возвратов 25 и показатель revenue 100.5. Результат следует показать на рисунке 5.12.

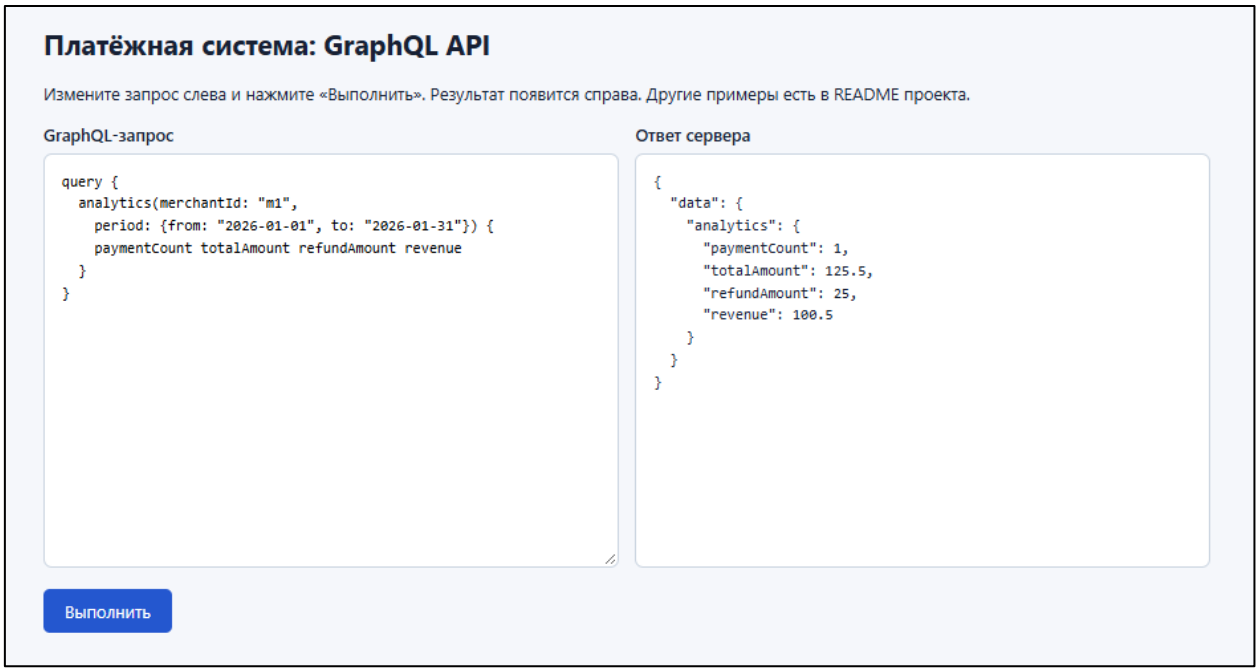


Рисунок 5.12 – Получение аналитических показателей

Таким образом, подготовлены примеры для проверки заявленных сценариев чтения. Снимки результатов выполнения приложения на демонстрационных данных дополняют описание реализации в предыдущих разделах.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы создана базовая структура платёжной системы на Java. Выделены модуль GraphQL API и модули внутренних сервисов, подготовлены схемы PostgreSQL и демонстрационные данные. Реализованы операции чтения объектов, списков и связанных данных, а также обработка отсутствующих записей.

Для взаимодействия gateway с сервисами определён интерфейс клиента и реализована его mock-версия без сетевых вызовов. Проверены получение платежа, связанных данных, фильтрация списка и чтение аналитических записей. Docker Compose обеспечивает сборку и запуск приложения и базы данных одной командой. Полученная структура может быть использована как основа для последующей реализации реального межсервисного взаимодействия.